

PAPER

IoT Modeling and Verification: From the CaIT Calculus to UPPAAL*

Ningning CHEN^{†a)}, *Nonmember* and Huibiao ZHU^{†b)}, *Member*

SUMMARY With the support of emerging technologies such as 5G, machine learning, edge computing and Industry 4.0, the Internet of Things (IoT) continues to evolve and promote the construction of future networks. Existing work on IoT mainly focuses on its practical applications, but there is little research on modeling the interactions among components in IoT systems and verifying the correctness of the network deployment. Therefore, the Calculus of the Internet of Things (CaIT) has previously been proposed to formally model and reason about IoT systems. In this paper, the CaIT calculus is extended by introducing broadcast communications. For modeling convenience, we provide explicit operations to model node mobility as well as the interactions between sensors (or actuators) with the environment. To support the use of UPPAAL to verify the temporal properties of IoT networks described by the CaIT calculus, we establish a relationship between timed automata and the CaIT calculus. Using UPPAAL, we verify six temporal properties of a simple “smart home” example, including Boiler On Manually, Boiler Off Automatically, Boiler On Automatically, Lights On, Lights Mutually, and Windows Simultaneously. The verification results show that the “smart home” can work properly.

key words: *IoT, CaIT calculus, timed automata, UPPAAL*

1. Introduction

The emergence of the Internet of Things (IoT) shifts the development focus of the Mobile Internet from networks of “end users” to networks of “objects”, where “objects” can autonomously interact for a common goal. The “things” contained in IoT systems can be RFID tags (identifying objects), sensors (capturing the changes in the environment), and actuators (providing information to the environment). Recently, IoT has been gradually combined with other technologies such as cloud computing [1], artificial intelligence [2], vehicle networks [3], etc. Relevant research results have been applied in many fields, including environmental monitoring, health care, smart environment, and

transportation. However, as IoT systems become more and more complex, their quality and reliability are difficult to guarantee, development costs are getting higher and higher, and new security issues are constantly emerging. Fortunately, the use of formal methods in the design and development of IoT systems can effectively alleviate the above problems.

Both model-checking techniques and process algebras are powerful tools for formal researches. Krishna et al. adopted Business Process Model Notation (BPMN) to express the underlying IEC 61499 specification [4] and developed an IoT Composer tool to support the development of IoT applications [5]. All of these studies apply model-checking approaches for verification, i.e., systems are checked against formulas given in temporal logic [6]. Differently, process algebras focus on taking (higher-level) system descriptions as specifications for (more detailed) system specifications, which are useful for modeling a programming abstraction that concentrates, especially for challenges in developing distributed and concurrent systems [7]. Both model-checking approaches and process algebras have their strengths and can be mutually beneficial. Process algebras are more abstract and expressive, which can provide a complete and accurate formal specification of systems [8]. For instance, the equivalence and refinement relations employed in process algebras are compositional. Hence, one (smaller, more analyzable) equivalent system can be substituted for another (larger, more complex) one, improving the efficiency of model-checking [7]. Using model-checking techniques, especially various tools, it is possible to mechanize the systems described by a process calculus and further verify their properties. Thus, some studies combine model-checking techniques with process algebras [9], [10]. Jackson translated Timed CSP processes into time automata to verify the logical properties of CSP processes using timed automata techniques [9]. Ouaknine et al. have proved the equivalence between Timed CSP and closed timed automata [10]. Inspired by these works, we adopt process algebras and model-checking techniques together to explore IoT systems.

Some works have been done to model and verify IoT systems from the perspective of process calculi. Lanese et al. proposed the first calculus for IoT systems, called IoT-calculus, to capture the interactions between sensors, actuators, and computer processes to provide services for “end users” [11]. To express a discrete notion of time and verify the temporal properties of IoT systems, Lanotte et

Manuscript received December 16, 2022.

Manuscript revised April 18, 2023.

Manuscript publicized June 2, 2023.

[†]The authors are with the Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai, China.

*This work was partially supported by the National Key Research and Development Program of China (No. 2022YFB3305102), the National Natural Science Foundation of China (Grant Nos. 62032024), the “Digital Silk Road” Shanghai International Joint Lab of Trustworthy Intelligent Software (No. 22510750100), Shanghai Trusted Industry Internet Software Collaborative Innovation Center, and the Dean’s Fund of Shanghai Key Laboratory of Trustworthy Computing (East China Normal University).

a) E-mail: nnchen@stu.ecnu.edu.cn

b) E-mail: hbzhu@sei.ecnu.edu.cn (Corresponding author)

DOI: 10.1587/transinf.2022EDP7223

al. presented the CaIT calculus with the reduction semantics and a labeled transition system [12]. Furthermore, the CaIT calculus is equipped with a standard notion of labelled bisimilarity, which provides a coinductive characterisation of the contextual equivalence [12]. The notion of labeled bisimilarity can facilitate the usage of model-checking techniques [7]. However, both IoT-calculus and the CaIT calculus only support point-to-point communications, not broadcast communications which are common in IoT systems. Thus, we enrich the CaIT calculus with broadcast primitives.

In recent years, a significant amount of work has been achieved in formal modeling and analyzing broadcast behavior from the views of process algebras. The Calculus for Broadcasting Systems (CBS) [13] is the first calculus to take broadcast as the communication primitive. $b\pi$ -calculus [14] extends π -calculus by introducing a broadcast paradigm. However, they only focus on global communication, i.e., allowing all nodes to receive broadcast messages at the same time, which makes the information vulnerable to eavesdropping by attackers. Thus, we refer to process calculi for local broadcast (such as $CBS\#$, CWS , and w -calculus) to further limit the transmission area of broadcast messages [15]–[17]. Consequently, our calculus requires that only adjacent nodes in IoT systems listening to the corresponding channel can receive the broadcast.

The CaIT calculus is a fully abstract semantic theory, proposed to specify and reason on IoT applications [12]. Using the CaIT calculus, various IoT systems can be described as similar abstract models, which is much clearer and more concise than describing them by directly applying model checking techniques [7], [12]. Furthermore, the CaIT calculus can support the future exploration of the relationship between IoT systems and has been used to explore the equivalence between IoT systems [12]. Whereas, analyzing the relationships between multiple IoT systems directly using model checking techniques is much more complicated. However, there are currently no tools that support the verification and analysis of IoT systems modeled by the CaIT calculus. To further use tools to mechanize the IoT systems modeled by the CaIT calculus and explore their temporal properties, we establish a relationship between the CaIT calculus and timed automata. Guided by this relationship, we construct a series of timed automata for the CaIT calculus, involving basic commands (or statements) and parallel compositions. Numerous tools can mechanize these automata, among which UPPAAL supports an intuitive and understandable graphical representation and is widely used to explore channel communications.

The main contributions of this paper include the following three aspects:

- The CaIT calculus is extended by applying broadcast communications as well as adding explicit operations for describing node mobility and interactions between sensors (or actuators) with the environment.
- We build the relationship between the CaIT calculus

and timed automata, involving each basic command (or statement) and the parallel composition. We implement all of these automata in UPPAAL, and further explore the networks of timed automata.

- Based on this relationship, an example “smart home” is modeled. Using UPPAAL, we verify its six temporal properties.

The rest of this paper is organized as follows. Section 2 introduces the CaIT calculus and presents a simple “smart home”. Section 3 describes the relationship between the CaIT calculus and timed automata. Section 4 gives a brief introduction to UPPAAL and the implementation of the CaIT calculus in UPPAAL, which allows us to achieve the “smart home” and verify some temporal properties. Conclusion and future work are given in Sect. 5.

2. The CaIT Calculus

2.1 Syntax

The original CaIT calculus [12] for IoT only uses point-to-point communications. Here, we extend the original CaIT with broadcast communications. In addition, we add explicit processes for formalizing node mobility, and the interactions between sensors (or actuators) with the environment. But for simplicity, we still use the name the CaIT calculus here. The syntax is shown in Table 1, where the extensions are in bold. It has a three-level structure: the upper level shows processes, the middle one illustrates the body of a node, and the lower level describes networks. Now, we show the details of the CaIT calculus from the three levels.

Table 1 The syntax of the CaIT calculus

<i>Process</i>		
P, Q	$::=$	nil Termination
	$ $	$skip$ Skip
	$ $	$P Q$ Parallel Composition
	$ $	$P \sqcap Q$ Nondeterministic Choice
	$ $	$[b]P, Q$ Conditional
	$ $	$!\langle v_2 \rangle^{bc}; P$ Broadcast Output
	$ $	$\rho; P$ Intra-node Action
	$ $	$[\pi; P]Q$ Action with Timeout
	$ $	X Process Variable
	$ $	$fix X \cdot P$ Recursion
ρ	$::=$	σ Delay
	$ $	$@h$ Querying Location
	$ $	$s?y$ Sensor Reading
	$ $	$a!v$ Actuator Writing
π	$::=$	$c!\langle v_1 \rangle$ Point-to-Point Output
	$ $	$c?(x_1)$ Point-to-Point Input
	$ $	$?(x_2)^{bc}$ Broadcast Input
	$ $	$move_k$ Migration
<i>Body</i>		
B, B_1	$::=$	$B B_1$ Parallel Composition
	$ $	P Process
	$ $	$(s \leftarrow v_3); B$ Sensor
	$ $	$(a \rightarrow v_4); B$ Actuator
<i>Network</i>		
N, M	$::=$	0 Empty Network
	$ $	$M N$ Parallel Composition
	$ $	$(vc')M$ Channel Restriction
	$ $	$n[I \bowtie B]_I^u$ Node

Process Level:

- (1) nil means that this process terminates.
- (2) $skip$ does nothing and has no effect.
- (3) $P||Q$ is the parallel composition of processes.
- (4) $P \sqcap Q$ is the nondeterministic choice.
- (5) $[b]P, Q$ represents the conditional choice, where b has the form $w = w'$, and w and w' are expressions. If Boolean expression b is true (that is, w is equal to w'), process P executes; otherwise, process Q works.
- (6) $!\langle v_2 \rangle^{bc}; P$ denotes that a node sends value v_2 via broadcast channel bc , and then P runs, where v_2 can be the value of a variable, a constant or the value constructed from values and functions. When describing actual IoT scenarios, values can be any specific data type, such as integer, character, generic, boolean, etc. After sending out v_2 , only nodes within the communication range of bc and listening to bc can receive it.
- (7) $\rho; P$ represents intra-node actions, where $\rho \in \{\sigma, @h, s?y, a!v\}$. $\sigma; P$ denotes that after waiting for one time unit, process P works. $@h; P$ queries the current physical location of the node in which this process is running. The physical location information is available for the subsequent process P . $s?y; P$ reads a value from sensor s . $a!v; P$ writes the value v into the actuator a . Especially, $a!v; P$ and $(s \leftarrow v_3); B$ can update Γ , e.g., the value of actuator a can be changed to be v due to the execution of $a!v; P$.
- (8) $[\pi; P]Q$ indicates that some actions execute with timeouts, where $\pi \in \{c!\langle v_1 \rangle, c?(x_1), ?(x_2)^{bc}, move_k\}$. For commands $[c!\langle v_1 \rangle; P]Q$ and $[c?(x_1); P]Q$, if processes can communicate within one time unit via point-to-point channel c , then process P carries out. Otherwise, Q gains control after waiting for one time unit. Similarly, we define $[(x_2)^{bc}; P]Q$ to show that if this process can receive a value via broadcast channel bc before one time unit, the subsequent behavior is like P . Otherwise, after one time unit, Q performs. Next, we give the details of the migration process $[move_k; P]Q$. After delaying one time unit, if the node can move to the destination k from the current location, $[move_k; P]Q$ executes and the following process P runs at the new location k ; otherwise, Q starts to work at its original location. Consequently, the CaIT calculus allows one node to move with the migration process to obtain its desired data. $\pi; P$ means that P will start if and only if π is complete.
- (9) X represents the process variable.
- (10) $fix X \cdot P$ is the guarded recursion, which is similar to the recursion construction proposed by Hoare (i.e., $\mu X : A \cdot F(X)$) [18], except that $fix X \cdot P$ takes time into account.

Body Level:

- (1) $B||B_l$ stands for the parallel composition.
- (2) Process P denotes the ongoing computations.
- (3) $(s \leftarrow v_3); B$ models a sensor whose name is s , and its current value is v_3 . A node can read a particular piece of information from the environment using a sensor and then make it available to the remaining node body B .
- (4) $(a \rightarrow v_4); B$ illustrates the behavior of an actuator a , representing that a node can provide a value v_4 to the en-

vironment through this actuator. To ensure that nodes exchange information with the environment in time, we suppose that $(s \leftarrow v_3)$ and $(a \rightarrow v_4)$ are both instantaneous actions.

Network Level:

- (1) 0 denotes an empty network.
- (2) $M||N$ is the parallel composition of networks.
- (3) $(\nu c')M$ shows that channel c' is private for the network M .
- (4) $_n[\Gamma \bowtie B]_l^u$ represents a network node, where n is the node ID, and B is the node body. The physical interface is represented by Γ , which maps the names of sensors and actuators to values. For security aims, each Γ is private for a node. In other words, given a node $_n[\Gamma \bowtie B]_l^u$, sensors in Γ can be read only by node n . Similarly, the values of actuators in Γ can be modified only by this node. l records the current location of node n . $u \in \{s, m\}$ is defined to distinguish stationary nodes (formed by $u = s$) and mobile nodes (denoted by $u = m$).

2.2 The Operational Semantics of the CaIT Calculus

Here, we present the operational semantics for our extensions in Table 2. To define the rules precisely, we give the following three kinds of transitions, which also cover the original CaIT calculus (see [12] for the corresponding rules).

Process: $P \xrightarrow{\lambda} Q$, where $\lambda \in \{\sigma, c!\langle v \rangle, c?(v), move_k, !\langle v \rangle^{bc}, ?(v)^{bc}, \tau\}$.

Body: $B \xrightarrow{\iota} B'$, where $\iota \in \{s \leftarrow v_3, a \rightarrow v_4\} \cup \lambda$.

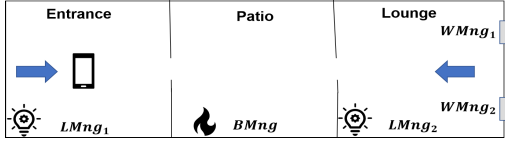
Network: $M \xrightarrow{\varsigma} N$, with $\varsigma \in \{\sigma, \tau, !\langle v \rangle^{bc} @ n, ?(v)^{bc} @ n, c!\langle v \rangle @ n, c?(v) @ n, (s \leftarrow v_3) @ n, (a \rightarrow v_4) @ n\}$.

Now, we give the explanations for the rules which have been listed in Table 2.

- Rule (**BSenP**) denotes sending messages via the broadcast channel bc . After that, process P executes.
- Rule (**BRecP**) describes receiving messages via bc before one time unit, then the process switches to P .
- Rule (**MoveP**) represents the execution of the migration action. After that, this process behaves as process P .
- Rule (**SensorB**) shows that after sensing information from the environment through a sensor, the remaining node body B executes. Similarly, rule (**ActuatorB**) describes providing information to the environment with an actuator.
- Rule (**BSenN**) models that after the node n broadcasts messages via bc , it switches to the remaining node body B' . Rule (**BRecN**) can be explained similarly.
- Rule (**BCom**) describes broadcast communications as limited by the communication range, that is, two networks can communicate successfully only if the distance between them is within the communication range used by them.
- Rules (**ResB**) and (**Res**) display the effect of closing the scope of a broadcast channel. In other words, a restricted channel cannot occur in a transition label.
- Rule (**Move**) describes that a node n moves to its destination k successfully from location l if the distance between l

Table 2 The operational semantics of CaT calculus

Process:		
(BSenP) $\frac{-}{!(v)^{bc}; P \xrightarrow{!(v)^{bc}} P}$	(BRecP) $\frac{-}{[?(x)^{bc}; P]Q \xrightarrow{?(v)^{bc}} P[v/x]}$	(MoveP) $\frac{-}{[move\ k; P]Q \xrightarrow{move\ k} P}$
Body:		
(SensorB) $\frac{-}{(s \leftarrow v_3); B \xrightarrow{s \leftarrow v_3} B}$	(ActuatorB) $\frac{-}{(a \rightarrow v_4); B \xrightarrow{a \rightarrow v_4} B}$	
Network:		
(BSenN) $\frac{B \xrightarrow{!(v)^{bc}} B'}{n[\Gamma \bowtie B]_l^u \xrightarrow{!(v)^{bc} @ n} n[\Gamma \bowtie B']_l^u}$	(BRecN) $\frac{B \xrightarrow{?(v)^{bc}} B'}{n[\Gamma \bowtie B]_l^u \xrightarrow{?(v)^{bc} @ n} n[\Gamma \bowtie B']_l^u}$	
(BCom) $\frac{M \xrightarrow{!(v)^{bc} @ m} M', N \xrightarrow{?(v)^{bc} @ n} N', Dis(n, m) \leq Rng(bc)}{M N \xrightarrow{!(v)^{bc} @ m} M' N'}$	(ResB) $\frac{M \xrightarrow{S} M', S \notin \{!(v)^{bc} @ n, ?(v)^{bc} @ n\}}{(vbc)M \xrightarrow{S} (vbc)M'}$	
(Move) $\frac{B \xrightarrow{\sigma} B', n[\Gamma \bowtie B]_l^m \xrightarrow{\tau} n[\Gamma \bowtie B']_k^m, Dis(k, l) \leq \delta, B' \xrightarrow{move\ k} B''}{n[\Gamma \bowtie B]_l^m \xrightarrow{\sigma} n[\Gamma \bowtie B']_k^m}$	(Res) $\frac{M \xrightarrow{!(v)^{bc} @ n} M'}{(vbc)M \xrightarrow{\tau} (vbc)M'}$	
(Sensor1) $\frac{\Gamma(s) \neq v_3, v_3 \in dom(s), B \xrightarrow{s \leftarrow v_3} B', \Gamma' ::= \Gamma[v_3/\Gamma(s)]}{n[\Gamma \bowtie B]_l^u \xrightarrow{(s \leftarrow v_3) @ n} n[\Gamma' \bowtie B']_l^u}$	(Sensor2) $\frac{I(s) = v_3, B \xrightarrow{s \leftarrow v_3} B'}{n[\Gamma \bowtie B]_l^u \xrightarrow{\tau} n[\Gamma \bowtie B']_l^u}$	
(Actuator) $\frac{v_4 = \Gamma(a), B \xrightarrow{a \rightarrow v_4} B'}{n[\Gamma \bowtie B]_l^u \xrightarrow{(a \rightarrow v_4) @ n} n[\Gamma \bowtie B']_l^u}$		

**Fig. 1** Smart home inspired by [12]

and k is less than or equal to a given value δ .

- Rules **(Sensor1)** and **(Sensor2)** describe the sensing information v_3 using a sensor s , distinguishing whether the value of the sensor is changed or not.
- Rule **(Actuator)** provides information to the environment.

2.3 Smart Home

In this subsection, we give a simple “smart home”, as shown in Fig. 1. Compared with the previous work [12], we add some processes to illustrate node mobility and broadcast communications. Especially, the application of broadcast communication allows users to control multiple devices simultaneously by sending only one command. The whole system is the parallel composition of the smart home *Home* and the smart phone *Phone*.

The house is composed of an entrance and a lounge which are separated by a patio, where $loc = \{entrance, patio, lounge\}$. In the entrance and the lounge, each one has a light (actuator $light_j$) managed by their light manager processes $LMng_j$, where $j \in \{1, 2\}$. In the lounge, there are two windows (actuator win_i) controlled by their window manager process $WMng_i$, where $i \in \{1, 2\}$. In the patio, there is a boiler (actuator $boiler$) governed by the boiler manager process $BMng$. *Phone* is a mobile node containing four processes: the migrating process *Move*, the boiler control process *BCtrl*, the light control process *LCtrl* and the window control process *WCtrl*. There are also some sensors to

Table 3 Smart home

System Modeling:
$Sys =_{df} Home \parallel Phone$
$Home =_{df} l_1 \parallel l_2 \parallel l_3$
$l_1 =_{df} n_1[\Gamma_1 \bowtie light_1 \parallel LMng_1]_{l_1}^{s_{entrance}}$
$l_2 =_{df} n_2[\Gamma_2 \bowtie temp \parallel boiler \parallel BMng]_{l_2}^{s_{patio}}$
$l_3 =_{df} n_3[\Gamma_3 \bowtie win_1 \parallel win_2 \parallel light_2 \parallel LMng_2 \parallel WMng_1 \parallel WMng_2]_{l_3}^{s_{lounge}}$
$Phone =_{df} np[\Gamma_P \bowtie bsensor \parallel wsensor \parallel PhoneP]_{l_P}^{m_{loc}}$
Actuator and Sensor Modeling:
$temp =_{df} (temp \leftarrow temperature)$
$boiler =_{df} (boiler \rightarrow \Gamma_B(boiler))$
$light_j =_{df} (light_j \rightarrow \Gamma_j(light_j)), \text{ for } j \in \{1, 2\}.$
$win_i =_{df} (win_i \rightarrow \Gamma_2(win_i)), \text{ for } i \in \{1, 2\}.$
$bsensor =_{df} (bsensor \leftarrow mode)$
$wsensor =_{df} (wsensor \leftarrow command)$
Process Modeling:
$PhonP =_{df} Move \parallel BCtrl \parallel LCtrl \parallel WCtrl$
$Move =_{df} fix\ X \cdot @ (h_1); [h_1 = entrance] \text{ move_patio}; X]X,$ $[h_1 = lounge] \text{ move_patio}; X]X,$ $[move_lounge; X]X \sqcap [move_entrance; X]X$
$BCtrl =_{df} fix\ X \cdot bsensor?z; [b!(z); \sigma; X]X$
$LCtrl =_{df} fix\ X \cdot @ (h_2); [h_2 = entrance] [c_1!(on); \sigma; X]X,$ $[h_2 = lounge] [c_2!(on); \sigma; X]X; \sigma; X$
$WCtrl =_{df} fix\ X \cdot wsensor?m; [on]^w; \sigma; X$
$BMng =_{df} fix\ X \cdot [b?(x_1); [x_1 = man] boiler!on; \sigma; Manual, TCtrl] TCtrl$
$Manual =_{df} fix\ Y \cdot b?(y); [y = auto] X; \sigma; Y$
$TCtrl =_{df} temp?te; [te < \theta] boiler!on; \sigma; X, boiler!off; \sigma; X$
$LMng_j =_{df} fix\ X \cdot [c_j?(x_2); light_j!on; \sigma; X] light_j!off; X$
$WMng_i =_{df} fix\ X \cdot [?(x_3)^w; win_i!on; \sigma; X] win_i!off; X$

support the operation of “smart home”, including *bsensor*, *wsensor*, and *temp*.

Based on the CaT calculus, the detailed description of “smart home” is shown in Table 3. We suppose that the transmission range of each channel is infinite. The trajectory of *Phone* is illustrated by process *Move*. Users set the mode of the boiler with the phone touchscreen button (sensor *bsensor*). When *Phone* senses these commands, it forwards them to *BMng* via point-to-point channel *b*. Once

the boiler is set to manual mode, the boiler will be turned on within one time unit. If it is in automatic mode, the boiler will be adjusted within one time unit according to the current physical temperature obtained by sensor *temp*. When *Phone* is located at *entrance* or *lounge*, *LCtrl* obtains its current location information and sends commands to the light manager process at the current location. Then the corresponding light manager process turns on its light. User governs windows using the phone touchscreen button (sensor *wsensor*). Via broadcast channel *w*, *WCtrl* can control *WMng₁* and *WMng₂* to open or close two windows simultaneously.

3. From the CaIT Calculus to Timed Automata

CaIT is a timed process calculus with a discrete notion of time, as it provides a special timing action σ to synchronize global time, and equips some actions (e.g., communications) with timeouts to control their execution [12]. Thus, CaIT can be mapped into timed automata. Inspired by the work [9], [10], [19]–[21], we explore the transformation from the CaIT calculus to timed automata, which enables us to use UPPAAL to verify some properties of IoT networks described by the CaIT calculus.

3.1 Overview of Timed Automata

We now present the basic definitions of the syntax and semantics for a timed automaton [20]–[22]. We first give some notations. C is the set of clocks, each of which is a real value variable. A clock constraint is a conjunctive formula of atomic constraints of the form $x \leq_s z$ or $x - y \leq_s z$, where $x, y \in C, z \in \mathbb{R}$ (real number set) and $\leq_s \in \{<, \leq, =, \geq, >\}$. The set of clock constraints is $B(C)$. Σ is the set of actions (or alphabet).

Definition 1. (Timed Automata (TA)) A timed automaton is a tuple $A = \langle L, l_0, E, I \rangle$, where

- L is the set of locations.
- $l_0 \in L$ is the initial location.
- $E \subseteq L \times B(C) \times \Sigma \times 2^C \times L$ is the set of edges, where each edge describes the transition between two locations with a guard, an action, and a set of clocks to be reset.
- $I : L \rightarrow B(C)$, which assigns invariants to locations.

We now introduce more details of edges (i.e., $E \subseteq L \times B(C) \times \Sigma \times 2^C \times L$), where an edge can be described as $\langle l, g, a, w, l' \rangle$.

- (1) $l \xrightarrow{g, a, w} l'$ is a shorthand notation for $\langle l, g, a, w, l' \rangle$, which simulates the transition from location l to location l' . Here, $g \in B(C)$ is a guard, $a \in \Sigma$ is an action, and $w \subseteq C$ is the set of clocks that need to be reset to 0.
- (2) When this transition happens, only clocks in w are reset to 0, and other clocks in $C \setminus w$ agree with u , that is, $[w \mapsto 0]u$. Similarly, $[w_1 \cup w_2 \mapsto 0]u$ resets all clocks in w_1 and w_2 .

We now define clock valuations, each of which is a function $u : C \rightarrow \mathbb{R}^+$ from the set of clocks to the non-negative reals. Some notations about clock valuations are as follows.

- (1) $\mathbb{R}^+^C$ is the set of all clock valuation functions. $u_0(x) = 0$ initializes all clocks in C , where x stands for any clock in C .
- (2) $u + d$ maps each clock x in C to the value $u(x) + d$, where $d \in \mathbb{R}^+$ is a non-negative real.
- (3) $u \models g$ means that the clock values assigned by u satisfy the guard g . Similarly, $u \models I(l)$ denotes that u satisfies the invariants of location l .

Definition 2. (Semantics of TA) Let $\langle L, l_0, E, I \rangle$ be a timed automaton, with the clock set C and the action set Σ . Its semantics is defined as a labelled transition system $\langle S, s_0, \rightsquigarrow \rangle$, where

- $S \subseteq L \times \mathbb{R}^+^C$ is the set of states. The state of this automaton is (l, u) , meaning that its current location and the current clock valuation are l and u , respectively.
- $s_0 = (l_0, u_0)$ is the initial state.
- $\rightsquigarrow \subseteq S \times \{\mathbb{R}^+ \cup \Sigma\} \times S$ is the transition relation such that

– **Delay Transition:**

$\langle l, u \rangle \xrightarrow{d} \langle l, u + d \rangle$,
if $\forall d' \in [0, d] : u + d' \models I(l)$, where $d, d' \in \mathbb{R}^+$.

– **Action Transition:**

$\langle l, u \rangle \xrightarrow{a} \langle l', u' \rangle$,
if there exists an edge $l \xrightarrow{g, a, w} l'$, $u \models g$,
 $u' = [w \mapsto 0]u$, and $u' \models I(l')$.

3.2 The Semantics of Timed Automata Networks

A timed automaton network is a parallel composition of several timed automata, with a common clock set C and a common action set Σ . If a timed automaton network contains n timed automata, it can be described as $A_1 \| A_2 \| \dots \| A_n$, where $A_i = \langle L_i, l_i^0, E_i, I_i \rangle$, $1 \leq i \leq n$, and $i, n \in \mathbb{N}^+$ (positive integers set). Compared with the traditional operational semantics of a network of timed automata [23], we discuss not only the rules for point-to-point communications, but also the rules for broadcast communications and channel restriction.

Before giving the semantics of a network of timed automata, we give the following notations.

- $\vec{l} = (l_1, \dots, l_n)$ is a location vector, where l_i is the i th element of $\vec{l}$, for $i \in \mathbb{N}^+$.
- $I(\vec{l}) = \bigwedge_{i=1}^n I_i(l_i)$ integrates all invariants functions into a common function over location vectors.
- $\vec{l}[l'_i/l_i]$ replaces the i th element l_i of $\vec{l}$ with l'_i .
- $\vec{l}_N$ is the location vector for the time automata network built for the IoT network N .

Definition 3. (Semantics of a network of Timed Automata) Let a network of n timed automata be $A_1 \| \dots \| A_n$,

where $A_i = \langle L_i, l_i^0, E_i, I_i \rangle$, $1 \leq i \leq n$, and $i, n \in \mathbb{N}^+$. Its semantics is defined as a labelled transition system $\langle \bar{S}, \bar{s}_0, \rightsquigarrow \rangle$, where

- $\bar{S} \subseteq (L_1 \times \dots \times L_n) \times \mathbb{R}^{+C}$ is the set of states. Let $\langle \bar{l}, u \rangle$ be a state of the network of timed automata.
- $\bar{s}_0 = (\bar{l}_0, u_0)$ is the initial state, where $\bar{l}_0 = (l_1^0, \dots, l_n^0)$.
- $\rightsquigarrow \subseteq \bar{S} \times \{R^+ \cup \Sigma\} \times \bar{S}$ is the transition relation, such that

1. Delay:

$\langle \bar{l}, u \rangle \xrightarrow{d} \langle \bar{l}, (u + d) \rangle$,
if $\forall d' \in [0, d] : (u + d') \models I(\bar{l})$, where $d, d' \in \mathbb{R}^+$.

2. Pointt Communication:

$\langle \bar{l}, u \rangle \xrightarrow{\tau} \langle \bar{l}', u' \rangle$,
if there exists $l_j \xrightarrow{g_j, c!, w_j} l'_j$, and $l_i \xrightarrow{g_i, c?, w_i} l'_i$ ($j \neq i$), such that

- (1) $u \models (g_j \wedge g_i)$ means that u satisfies the guards of two edges.
- (2) $Dis(n_j, n_i) \leq Rng(c)$ holds. c is a point-to-point channel, and $Rng(c) > 0$.
- (3) $\bar{l}' = \bar{l}[l'_j/l_j, l'_i/l_i]$.

(4) $u' = [w_j \cup w_i \mapsto 0]u$ and $u' \models I(\bar{l}')$.

$Dis(n_j, n_i) \leq Rng(c)$ denotes that the distance between two nodes (i.e., n_j and n_i) is within the transmission range of channel c , where l_j and l_i are the current location of the automata of n_j and n_i , respectively. $j \neq i$ means that this is an inter-node communication. If the communication occurs in a node, it can be addressed by the τ -action rule (left column, page 6).

3. Broadcast Communication:

$\langle \bar{l}, u \rangle \xrightarrow{!bc} \langle \bar{l}', u' \rangle$,

if there exists an edge $l_j \xrightarrow{g_j, !bc, w_j} l'_j$

and m edges $l_{i_k} \xrightarrow{g_{i_k}, ?bc, w_{i_k}} l'_{i_k}$ for $1 \leq k \leq m$ and $k, i_k, m \in \mathbb{N}^+$ ($j \neq i_k$), such that

- (1) $l_{i_1}, l_{i_2}, \dots, l_{i_m}$ are ordered according to the automata ordering $A_1, \dots, A_n$.
- (2) $u \models (g_j \wedge \bigwedge_{k=1}^m g_{i_k})$ means that u satisfies the guards of edges $l_j, l_{i_1}, l_{i_2}, \dots, l_{i_m}$.
- (3) $\bigwedge_{k=1}^m Dis(n_j, n_{i_k}) \leq Rng(bc)$ holds. bc is a broadcast channel, and $Rng(bc) > 0$.
- (4) $\bar{l}' = \bar{l}[l'_j/l_j, l'_{i_1}/l_{i_1}, \dots, l'_{i_m}/l_{i_m}]$.

(5) $u' = [(w_j \cup (\bigcup_{k=1}^m w_{i_k})) \mapsto 0]u$ and $u' \models I(\bar{l}')$.

According to $b\pi$ -calculus [14], broadcast sending is never blocked. In other words, the output action executes as usual even if no receiver is within the communication range of bc . Therefore, we give the following **Broadcast Output** rule to describe the case at which only the broadcast output action occurs.

4. Broadcast Output: $\langle \bar{l}, u \rangle \xrightarrow{!bc} \langle \bar{l}', u' \rangle$,

if there only exists $l_j \xrightarrow{g_j, !bc, w_j} l'_j$ ($Rng(bc) > 0$) but no edges describing the matching input actions, such that

- (1) $u \models g_j$.
- (2) $\bar{l}' = \bar{l}[l'_j/l_j]$.

(3) $u' = [w_j \mapsto 0]u$ and $u' \models I(\bar{l}')$.

5. τ -action: $\langle \bar{l}, u \rangle \xrightarrow{\tau} \langle \bar{l}[l'_j/l_j], u' \rangle$,

if there exists $l_j \xrightarrow{g_j, \tau, w_j} l'_j$, $u \models g_j$, $u' = [w_j \mapsto 0]u$ and $u' \models I(\bar{l}[l'_j/l_j])$.

6. Channel Restriction: $\langle (vc')\bar{l}_N, u \rangle \xrightarrow{\tau} \langle (vc')\bar{l}_{N'}, u' \rangle$,

if there exists $l_j \xrightarrow{g_j, !c' / c'!, w_j} l'_j$ ($Rng(c') > 0, n_j \in N$), such that

(1) $u \models g_j$.

(2) $\bar{l}_{N'} = \bar{l}_N[l'_j/l_j]$.

(3) $u' = [w_j \mapsto 0]u$ and $u' \models I(\bar{l}')$.

where l_j is the current location of the automaton built for node n_j , and $n_j \in N$ means that n_j belongs to N .

Referring to the works [24]–[26], $(vc')N$ binds the broadcast channel c' to network N . As shown in Fig. 2, the output action occurring on c' (i.e., $!c'$) is replaced by the silent action (i.e., τ) when it exceeds N . As a result, the information transmitted on c' is invisible to nodes outside N , thus ensuring the communication security.

Structural Congruence:

$\langle \bar{l}_M, u_M \rangle \parallel \langle \bar{l}_N, u_N \rangle \equiv \langle [\bar{l}_M, \bar{l}_N], u_M \cup u_N \rangle$

Here, $\langle \bar{l}_M, u_M \rangle$ and $\langle \bar{l}_N, u_N \rangle$ are the current states of the automata networks built for IoT networks M and N , respectively. $[\bar{l}_M, \bar{l}_N]$ combines location vectors $\bar{l}_M$ and $\bar{l}_N$. And $u_M \cup u_N$ integrates clock valuations u_M and u_N .

3.3 Relating Basic Commands to Timed Automata

For a component $m[\Gamma \bowtie B]^\mu$ of a CaIT network, we construct a timed automaton, shown by $A = \langle L, l_0, E, I \rangle$ with a clock x . Initially, $L = \{m_0\}$, $l_0 = m_0$, $E = \emptyset$ and $I = \emptyset$. As mentioned in Sect. 2, the node body B can be a process P or some commands about sensors and actuators. The locations of the automaton are labeled using the node ID in which the node body B is running, and an index is starting from 0, to identify the locations in the automaton uniquely. Thus, m_0 indicates that we model a node body running in a node m . Depending on the structure of B , components L, E and I can be updated.

We assume that all the following components are executed in the node m . When they start running, the current $L = \{m_0, \dots, m_i\}$, where $i \geq 0$. For the sake of brevity, we only give definitions of timed automata associated with some representative commands here. Others are similar.

1. For $P = [c!(v); P_1]P_2$, we have

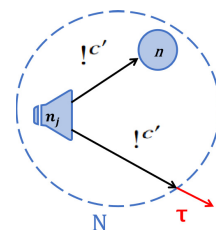


Fig. 2 Channel restriction $(vc')N$

- $L = L \cup \{m_{i+1}, m_{i+2}\};$
* Locations m_{i+1} and m_{i+2} are added to L . They respectively describe the two executions of process P , leading to either process P_1 or process P_2 .
 - $E = E \cup \{\langle l, x < 1, c!, x = 0, m_{i+1} \rangle\} \cup \{\langle l, x == 1, \tau, x = 0, m_{i+2} \rangle\};$
* If $i > 0$, it means that this automaton already has some edges, and P is lunched from a process Q , such as $Q = [c_1! \langle v \rangle; P]Q_1$. In other words, P is led by an action that has been modeled by an edge linking the last component m_f of Q . Then $l = m_f$ holds, denoting that P starts from m_f .
* Otherwise, $l = m_0$ holds, meaning that P begins from the initial location of the automaton.
The first new edge $\langle l, x < 1, c!, x = 0, m_{i+1} \rangle$ shows that if this point-to-point output action occurs within one time unit, this process terminates, and then P_1 gains the control. Another edge $\langle l, x == 1, \tau, x = 0, m_{i+2} \rangle$ denotes that this action cannot happen before one time unit. This process gives up, and P_2 works continually.
 - $I(l) = \{x \leq 1\}.$
* This invariant ensures that the point-to-point output action executes within one time unit.
2. For $P = [c?(x_1); P_1]P_2$, we have
- $L = L \cup \{m_{i+1}, m_{i+2}\};$
* We add locations m_{i+1} and m_{i+2} into L , which correspond to the two situations of process P , leading to either process P_1 or process P_2 .
 - $E = E \cup \{\langle l, x < 1, c?, \{x = 0, x_1 = v\}, m_{i+1} \rangle\} \cup \{\langle l, x == 1, \tau, x = 0, m_{i+2} \rangle\};$
* If $i > 0$, applying a similar reasoning process as for the point-to-point output action, $l = m_f$ holds.
* Otherwise, l is m_0 .
The first edge describes the case in which the point-to-point input action occurs before one time unit, and process P_1 executes continually. Otherwise, the second edge represents that this communication cannot happen within one time unit, and the next process is P_2 . To use the assignment $x_1 = v$ on the edge with $c?$, we suppose that channel c can only be used at most once for output actions in the transformed CaIT networks at any time. This requirement reflects somehow the global asynchronicity in the distributed environment, which has been explored in a large number of related process calculi.
 - $I(l) = \{x \leq 1\}.$
3. For $P = !\langle v_2 \rangle^{bc}; P_1$, we have
- $L = L \cup \{m_{i+1}\};$
* Only location m_{i+1} is added into L , denoting its execution. Then the control is passed to P_1 .
 - $E = E \cup \{\langle l, x == 0, !^{bc}, x = 0, m_{i+1} \rangle\};$
* If P is lunched from another process, that is $i > 0$, it holds that $l = m_f$ by adopting a similar argument proposed in the point-to-point output action. In order to distinguish from point-to-point, we use the symbol $!^{bc}$ to stand for an output action occurring on the broadcast

channel bc .

* Otherwise, $l = m_0$.

• $I(l) = \{x \leq 0\}.$

* We give this invariant to ensure that this process is executed immediately.

4. For $P = [?(x_2)^{bc}; P_1]P_2$, we have

• $L = L \cup \{m_{i+1}, m_{i+2}\};$

* We add m_{i+1} and m_{i+2} into L , leading to either process P_1 or process P_2 .

• $E = E \cup \{\langle l, x < 1, ?^{bc}, \{x = 0, x_2 = v\}, m_{i+1} \rangle\} \cup \{\langle l, x == 1, \tau, x = 0, m_{i+2} \rangle\};$

* If the execution of P is launched from another process, with a similar argument as for the point-to-point output action, we can obtain $l = m_f$.

* Otherwise, l is equal to m_0 .

The first edge describes that this broadcast input process happens within one time unit, then P_1 works after that. If not, this process is modeled by another edge, leading to P_2 . To support the application of the assignment $x_2 = v$ and $?^{bc}$ in the edge, at any time, we similarly require that the broadcast channel bc can be applied at most once for output actions.

• $I(l) = \{x \leq 1\}.$

5. For $P = [move_k; P_1]P_2$, we gain

• $L = L \cup \{m_{i+1}, m_{i+2}\};$

* m_{i+1} and m_{i+2} are added to L . m_{i+1} and m_{i+2} indicate its two executions respectively, leading to either process P_1 or process P_2 .

• $E = E \cup \{\langle l, x == 1 \wedge Dis(loc, k) \leq \delta, \tau, x = 0, m_{i+1} \rangle\} \cup \{\langle l, x == 1 \wedge Dis(loc, k) > \delta, \tau, x = 0, m_{i+2} \rangle\};$

* If there is a process that launches P , with the similar argument as for the point-to-point output action, it is clear that $l = m_f$.

* Otherwise, $l = m_0$.

We construct two edges to model its two executions. After one time unit, according to its current physical location loc , the destination location k and its moving rate δ , this migration process can judge whether k is reachable. Using the first edge, we describe that the process P can move to k successfully. After one time unit, the following process P_1 starts to execute at new location k . The second one denotes that the destination is unreachable. The next process P_2 obtains the control at its original location after waiting for one time unit.

• $I(l) = \{x \leq 1\}.$

6. For $P = s?y; P_1$, we have

• $L = L \cup \{m_{i+1}\};$

• $E = E \cup \{\langle l, x == 0, \tau, \{x = 0, y = \Gamma(s)\}, m_{i+1} \rangle\};$

* If $i > 0$, it holds that $l = m_f$.

* Otherwise, $l = m_0$.

• $I(l) = \{x \leq 0\}.$

7. For $B = (s \leftarrow v_3); B_1$, we have

• $L = L \cup \{m_{i+1}, m_{i+2}\};$

* New locations m_{i+1} and m_{i+2} are added into L , representing its two executions.

- $E = E \cup \{\langle l, x == 0 \wedge \Gamma(s) \neq v_3, \tau, \{x = 0, \Gamma(s) = v_3\}, m_{i+1} \rangle\} \cup \{\langle l, x == 0 \wedge \Gamma(s) == v_3, \tau, x = 0, m_{i+2} \rangle\}$;
 * If $i > 0$, we obtain $l = m_f$ with a similar reasoning process for point-to-point output action.
 * Otherwise, $l = m_0$.

The first edge encodes that the newly obtained value v_3 immediately replaces the value of s . If its value is already v_3 , we don't need to change it, shown by another edge.

- $I(l) = \{x \leq 0\}$.

8. For $P = \text{nil}$ or $P = \text{skip}$, we have

- L, E and I keep unchanged, and the automaton stops executing.

9. For $P = \text{fix } X \cdot P_1$, we have

- L keeps the same.
- $E = E \cup \{\langle l, x == 0, \tau, x = 0, m_0 \rangle\}$;
 * If $i > 0$, using the similar argument constructed for the point-to-point output action, $l = m_f$ holds.
 * Otherwise, $l = m_0$.
 When this recursion action is executed, the automaton immediately returns to its initial node m_0 .
- $I(l) = \{x \leq 0\}$.

3.4 Relating Parallel Composition to Timed Automata

In this subsection, we construct timed automata for the parallel composition, denoted by $P_1 \parallel P_2 \parallel \dots \parallel P_n$, where $n > 1$. We assume that none of these processes has the parallel composition operator at its topmost level. As shown in Fig. 3, the parallel composition automaton has three layers:

- In the top layer, process P is waiting for execution in automaton A_1 . Broadcast channel bp is used to start all parallel processes in P simultaneously.
- In the second layer, there are two parts. In the first one, once the broadcast output action $! \langle v \rangle^{bp}$ occurs, only P_1 is kept in the original automaton A_1 for execution. Meanwhile, all broadcast input actions take place immediately, starting to build new automata for the processes following them. Thus, in the other part, there is a process automaton, which can be instantiated as a new process automaton for the specific process based on the process ID obtained from the top layer. For example, if it is triggered by $? \langle x_i \rangle^{bp}$, then it can construct a process automaton A_i for process P_i , where $i \in \{2, 3, \dots, n\}$. When we implement these process automata in UPPAAL, we creatively construct a table for each specific process to store their process bodies.
- In the third layer, there are a series of automata corresponding to basic commands, which have been constructed in Sect. 3.3. Each process automaton executes its process body by calling the corresponding automata of basic commands according to the command ID recorded in its table. Execution results are returned from the bottom layer to the second one to guide the execution of the remaining.

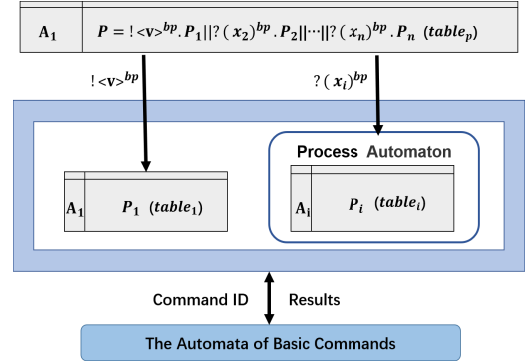


Fig. 3 The structure of parallel composition automaton

10. For $P = ! \langle v \rangle^{bp}; P_1 \parallel ? \langle x_2 \rangle^{bp}; P_2 \parallel \dots \parallel ? \langle x_n \rangle^{bp}; P_n$, $n > 1$, we can have

- $L = L \cup \{m_{i+1}\}$;
 * If process P is running in a node m , and there are already some locations in L (named as $m_0, \dots, m_i$), then add m_{i+1} into L to denote the execution of $! \langle v \rangle^{bp}$, leading to P_1 .
- $E = E \cup \{\langle l, !^b, x = 0, m_{i+1} \rangle\}$;
 * If the index $i > 0$, we can have $l = m_f$ similarly.
 * Otherwise, $l = m_0$.

The new edge models the execution of this broadcast output action, leading to P_1 . For each of the other processes P_j and $j > 1$, we build a new automaton, formed by $A_j = \langle l_j, n_{j0}, E_j, I_j \rangle$, where $l_j = \{m_0, m_1\}$, $n_{j0} = m_0$, $E_j = \{\langle m_0, ?^b, x = 0, m_1 \rangle\}$ and $I_j(m_0) = \emptyset$. Using the definition of P_j , these automata can be constructed.

- $I(l) = \{x \leq 0\}$.

3.5 Networks of Timed Automata

A network of timed automata is the parallel composition of a series of automata, formed by $A_1 \parallel A_2 \parallel \dots \parallel A_n$, where $n > 1$. None of these automata has a parallel combination operator at its topmost level [23]. We assume that this network is running in the automaton $A = \langle L, l_0, E, I \rangle$. Initially, $L = \{l_0\}$, $l_0 = l_0$, $E = \emptyset$ and $I = \emptyset$.

11. For $A = ! \langle v \rangle^b; A_1 \parallel ? \langle x_2 \rangle^b; A_2 \parallel \dots \parallel ? \langle x_n \rangle^b; A_n$, $n > 1$, we can have

- $L = L \cup \{l_1\}$;
 * l_1 is added into L , leading to the execution of the automaton A_1 .
- $E = E \cup \{\langle l, !^b, x = 0, l_1 \rangle\}$;
 * $l = l_0$.

The edge $\{\langle l, !^b, x = 0, l_1 \rangle\}$ models the execution of this broadcast output action, leading to A_1 . For each of other automata A_j and $j > 1$, we build a new automaton, formed by $A_j = \langle l_j, n_{j0}, E_j, I_j \rangle$, where $l_j = \{l_0, l_1\}$, $n_{j0} = l_0$, $E_j = \{\langle l_0, ?^b, x = 0, l_1 \rangle\}$ and $I_j(l_0) = \emptyset$. These automata can be built recursively.

- $I(l) = \{x \leq 0\}$.

3.6 Channel Restriction

As mentioned before, $(\nu c')M$ denotes that channel c' is private for the network M . Thus, output actions that take place on c' are invisible to nodes outside the network M . Based on the above, we can obtain the automata of M , simply denoted as $A_1 = \langle L, l_0, E, I \rangle$. To further get the automata A_2 of $(\nu c')M$, we give the following process to replace output actions occurring on c' with τ .

$$A_2 =_{df} \mathbf{Res}(A_1, c'),$$

where, $\mathbf{Res}(\langle L, l_0, E, I \rangle, c) =_{df} \forall e \in E \bullet e = \mathbf{Check}(e, c')$

$\mathbf{Check}(\langle l, g, a, w, l' \rangle, c') =_{df}$

$$\begin{cases} \langle l, g, \tau, w, l' \rangle, & \text{if } \mathbf{Output}(a) \wedge \mathbf{Chan}(a) = c'. \\ \langle l, g, a, w, l' \rangle, & \text{otherwise.} \end{cases}$$

3.7 Simulation

We now show how a CaIT network N is simulated by its associated timed automaton A_N in state $\langle \bar{l}_N, u_N \rangle$. We define a binary simulation relation $\mathcal{R}$ [27], [28] based on the operational semantics of CaIT calculus (see Sect. 2.2 and Appendix A) and the semantics of timed automata (see Sects. 3.1 and 3.2).

Definition 4. A simulation is defined as a binary relation $\mathcal{R}$ over CaIT networks and the timed automata, satisfying that if whenever $N \mathcal{R} (A_N, \langle \bar{l}_N, u_N \rangle)$,

- if $N \xrightarrow{\sigma} N'$, then there exists a timed automata $A_{N'}$ and a state $\langle \bar{l}_{N'}, u_{N'} \rangle$, such that $\langle \bar{l}_N, u_N \rangle \xrightarrow{d} \langle \bar{l}_{N'}, u_{N'} \rangle$ and $N' \mathcal{R} (A_{N'}, \langle \bar{l}_{N'}, u_{N'} \rangle)$, where $u_{N'} = (u_N + d)$ and $d = 1$.
- if $N \xrightarrow{!bc} N'$, then there exists a timed automata $A_{N'}$ and a state $\langle \bar{l}_{N'}, u_{N'} \rangle$, such that $\langle \bar{l}_N, u_N \rangle \xrightarrow{!bc} \langle \bar{l}_{N'}, u_{N'} \rangle$ and $N' \mathcal{R} (A_{N'}, \langle \bar{l}_{N'}, u_{N'} \rangle)$.
- if $N \xrightarrow{a \in \zeta \wedge a \notin \{!bc, \sigma\}} N'$, then there exists a timed automata $A_{N'}$ and a state $\langle \bar{l}_{N'}, u_{N'} \rangle$, such that $\langle \bar{l}_N, u_N \rangle \xrightarrow{\tau} \langle \bar{l}_{N'}, u_{N'} \rangle$ and $N' \mathcal{R} (A_{N'}, \langle \bar{l}_{N'}, u_{N'} \rangle)$.

where ζ is the action label set of CaIT networks (right column, page 3). $\square$

We use the notation $\leq_s$ to denote the largest simulation relation, i.e., $\leq_s = \bigcup \{ \mathcal{R} : \mathcal{R} \text{ is a simulation} \}$.

In a CaIT network N , all nodes wait together for one time unit when no (instantaneous) action is possible, as described by the **TimeParN** rule (see Appendix A). In its associated timed automaton A_N , all clocks are synchronously increased by one time unit in a continuous approach when no edge can be taken, which is supported by the timed automata theory [21], [23]. The simulation relations established for instantaneous actions are similar.

Definition 5. A CaIT network is well-formed if each of its channels can only be used to output actions at most once at any time, i.e., it satisfies the description of the global asynchrony of distributed systems in many process calculi.

Theorem 1. Given a well-formed CaIT network N , there exists a timed automaton A_N satisfying $N \leq_s (A_N, \langle \bar{l}_N, u_N \rangle)$.

Proof. Based on structural induction, we now prove this theorem by taking the parallel composition as an example. We

built the relation:

$$\mathbb{K} = \{ (M \| N, (A_M \| A_N, \langle \bar{l}_M; \bar{l}_N, u_M \cup u_N \rangle)) : \\ M \leq_s (A_M, \langle \bar{l}_M, u_M \rangle) \text{ and } N \leq_s (A_N, \langle \bar{l}_N, u_N \rangle) \}.$$

We next show that $\mathbb{K}$ is a simulation by analyzing the execution of case $M \| N \xrightarrow{\alpha} \hat{M}$.

- We assume that there exists a transition $M \| N \xrightarrow{\tau} \hat{M}$, which can be derived from the following two cases.

(1) This transition can be derived using the **PCom** rule (in Appendix A) with $\hat{M} = M' \| N'$, if $M \xrightarrow{c!(v)@m} M'$, $N \xrightarrow{c?(x)@n} N'$, $\text{Dis}(m, n) \leq \text{Rng}(c)$, and c is a point-to-point channel. Due to $M \leq_s (A_M, \langle \bar{l}_M, u_M \rangle)$ and $N \leq_s (A_N, \langle \bar{l}_N, u_N \rangle)$, there exist edges $l_m \xrightarrow{g_m, c!, u_m} l'_m$ and $l_n \xrightarrow{g_n, c?, u_n} l'_n$, where $\text{Dis}(m, n) \leq \text{Rng}(c)$, $u_M \models g_m$, and $u_N \models g_n$. Using **Point-to-Point Communication** rule of timed automata (right column, page 5), we can have $\langle \bar{l}_M; \bar{l}_N, u_M \cup u_N \rangle \xrightarrow{\tau} \langle \bar{l}_{M'}; \bar{l}_{N'}, u_{M'} \cup u_{N'} \rangle$ with $(\hat{M}, (A_{M'} \| A_{N'}, \langle \bar{l}_{M'}; \bar{l}_{N'}, u_{M'} \cup u_{N'} \rangle)) \in \mathbb{K}$, where $u_{M'} = [w_m \mapsto 0]u_M$, $u_{N'} = [w_n \mapsto 0]u_N$, $\bar{l}_{M'} = \bar{l}_M[l'_m/l_m]$, and $\bar{l}_{N'} = \bar{l}_N[l'_n/l_n]$. The symmetric case in which N sends a message to M via a point-to-point channel is similar.

(2) This transition can also be derived by the **ParN** rule (in Appendix A) with $\hat{M} = M' \| N$, where $M \xrightarrow{\tau} M'$. As $M \leq_s (A_M, \langle \bar{l}_M, u_M \rangle)$ and $N \leq_s (A_N, \langle \bar{l}_N, u_N \rangle)$, the proof is similar.

- For $M \| N \xrightarrow{a} \hat{M}$, where $a \in \{c!(v)@m, c?(x)@m, !\langle v \rangle^{bc}@m, ?\langle x \rangle^{bc}@m, (s \leftarrow v_3)@m, \sigma, (a \rightarrow v_4)@m, (\text{move } k)@m\}$, the proofs analyses are similar. $\square$

This theorem indicates the correctness of our transformation from a CaIT network to its associated timed automaton. Based on the reliable simulation relation, we can use UPPAAL to verify some properties of the IoT systems modeled by the CaIT calculus. If a property is valid for some IoT systems verified by applying UPPAAL, it is also valid for the models built for these IoT systems using the CaIT calculus.

4. Implementing CaIT in UPPAAL

In this section, we first give a brief introduction to UPPAAL. We then present the implementation of the CaIT calculus in UPPAAL, which allows us to further accomplish the “smart home” and verify its six temporal properties.

4.1 Overview of UPPAAL

UPPAAL is an integrated tool based on constraint-solving and on-the-fly techniques, which can be used to model, simulate, and verify real-time systems [29]. It is particularly suitable for formally analyzing systems with limited control structures, clocks, communication through channels, or shared variables [30]. As mentioned previously, CaIT is a timed process calculus with a discrete notion of time, which has been transformed into timed automata. Thus, UPPAAL is perfect for implementing the transformation between CaIT and timed automata as a tool based on timed

automata. UPPAAL has three main parts: a description language, a model-checker, and a simulator.

Description Language:

By defining templates with a set of parameters that can be any type, UPPAAL can model a real-time system as a network of timed automata. Taking a timed automaton as a directed graph, locations are its vertices connected by edges.

• **Locations.** There are three types of locations: initial locations, urgent locations, and committed locations.

(1) Initial Locations: Each template has one and only one initial location, which is described as a double circle.

(2) Urgent Locations: Urgent locations prohibit time pass, and an urgent location is described as a U in a circle.

(3) Committed Locations: Similarly, committed locations also freeze time, and a committed location is represented as a C in a circle. Once a template reaches a committed location, then the following transition must have an outgoing edge of at least one of the committed locations [31].

• **Edge.** Edges are noted with four types of labels: selections, guards, synchronizations, and updates.

(1) Selections: A selection label can bind a specific identifier to a random value belonging to a given type.

(2) Guards: A guard expresses conditions on the values of clocks and integer variables that must be satisfied before the edge can be taken [31].

(3) Synchronizations: The synchronization label is used to synchronize channel communication, which can be $c!$ (outputting) or $c?$ (inputting), where c stands for a channel.

(4) Updates: The update label is a comma-separated list of expressions, where these expressions only involve clocks, integer variables, and constants, and just assign integer values to clocks [22].

Model-checker:

By exploring the whole reachable state space, the model-checker can verify the reachability, safety, and liveness properties of a system. In UPPAAL, these properties are specified in a subset of Computational Tree Logic (CTL) [22].

Simulator:

During the early design (or modeling) stages, the simulator allows users to examine the possible dynamic behavior of a system in an interactive and graphic style.

4.2 The Implementation of CaIT in UPPAAL

By applying UPPAAL, all automata mentioned in Sect. 3 have been implemented [32]. Here, we take the broadcast input automaton (right column, page 7) as an example to show the implementation details of an automaton, as shown in Fig. 4. Other automata have been implemented similarly [32]. In Fig. 4, arrays X , $prolist$, $proread$, and $broc$ store process data, process bodies, the position of the next instruction to be executed by each process, and broadcast channels, respectively. Now, we will give the details.

- This automaton is triggered once it receives signals from a process automaton via channel $tr[3]$. Mean-

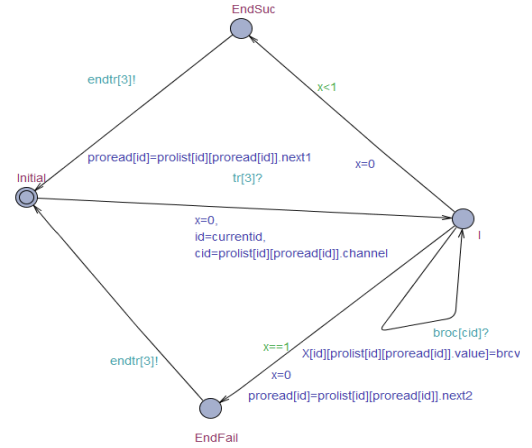


Fig. 4 The broadcast input automaton

while, its clock x is reset (i.e., $x = 0$), and it knows the process ID id that calls it and the channel $broc[cid]$ that it will use. Then it goes to location l .

- If it receives a value $brcv$ within one time unit (i.e., $x < 1$ marked in green), it switches to $EndSuc$ (i.e., m_{i+1}), and the process chooses the first branch (i.e., process P_1).
- Otherwise, when $x == 1$ (marked in green) holds, it converts to location $EndFail$ (i.e., m_{i+2}), meaning that the process chooses the alternative branch (i.e., process P_2).
- Finally, it needs to send a signal via channel $endtr[3]$ to tell the process automaton (see Sect. 3.4) that this action is complete.

4.3 Verification of Smart Home Using UPPAAL

Based on the automata of the CaIT calculus implemented in UPPAAL, we automatically simulate the example “smart home” and verify its six temporal properties.

4.3.1 Properties Verification

UPPAAL uses formulas $A <> \phi$, $A[\] \phi$, and $E <> \phi$ to check liveness properties, safety properties, and reachability properties, respectively [22], [31].

- $A <> \phi$ is true, indicating that all paths eventually reach a state satisfying the given predicate ϕ .
- $A[\] \phi$ is true, meaning that ϕ is satisfied for all reachable states in all paths. Further, $A[\] \text{not } \phi$ is true, showing that ϕ is always unsatisfied for all reachable states in all paths.
- $E <> \phi$ is true if and only if there exists one reachable state satisfying ϕ .

Property 1: Boiler On Manually (Liveness Property)

$$A <> \text{BroadcastInput3.EndSuc} \ \&\& \ \text{model} == 1 \\ \text{imply } \text{globclock} < 1 \ \&\& \ \text{boiler} == 1$$

This property denotes that if the boiler is in manual mode,

it should be switched on within one time unit. *globclock* is a global clock, which is added to our UPPAAL models to check whether actions occur within the given time units.

Property 2: Boiler Off Automatically (Liveness Property)

$$A \langle \rangle \text{ temp.EndSuc} \ \&\& \ \text{model} == 0 \ \&\& \ \text{temperature} \geq \text{threshold} \\ \text{imply globclock} < 1 \ \&\& \ \text{boiler} == 0$$

If the boiler is in automatic mode and the current temperature is higher than or equal to a given threshold, it should be switched off within one time unit.

Property 3: Boiler On Automatically (Liveness Property)

$$A \langle \rangle \text{ temp.EndSuc} \ \&\& \ \text{model} == 0 \ \&\& \ \text{temperature} < \text{threshold} \\ \text{imply globclock} < 1 \ \&\& \ \text{boiler} == 1$$

This property shows that the boiler should be switched on within one time unit, once it is in automatic mode and the current temperature is lower than a given threshold.

Property 4: Lights On (Reachability Property)

$$E \langle \rangle \text{ LightMng1.light} == 1 \\ E \langle \rangle \text{ LightMng2.light} == 1$$

The first formula shows that *light₁* can be turned on. The second one is defined for *light₂* similarly.

Property 5: Lights Mutually (Safety Property)

$$A[] \text{ not } (\text{LightMng1.light} == 1 \ \&\& \ \text{LightMng2.light} == 1)$$

This property means that this system cannot turn on the lights of two rooms at the same time.

Property 6: Windows Simultaneously (Reachability Property)

$$E \langle \rangle \text{ Window1.window} == 1 \ \&\& \ \text{Window2.window} == 1 \\ E \langle \rangle \text{ Window1.window} == 0 \ \&\& \ \text{Window2.window} == 0$$

This property ensures that this system can open or close two windows simultaneously.

4.3.2 Verification Results

As shown in Fig. 5, all properties are satisfied:

- **Property 1** is valid. It represents that the boiler can be switched on within one time unit, once it is set to manual mode.
- **Property 2** and **Property 3** are both satisfied, that is, if the boiler is in automatic mode, it can be precisely turned on or turned off within one time unit according to the current temperature.
- **Property 4** is valid. It means that both *light₁* and *light₂* can be turned on.
- **Property 5** is satisfied, indicating that two lights cannot be turned on at the same time.

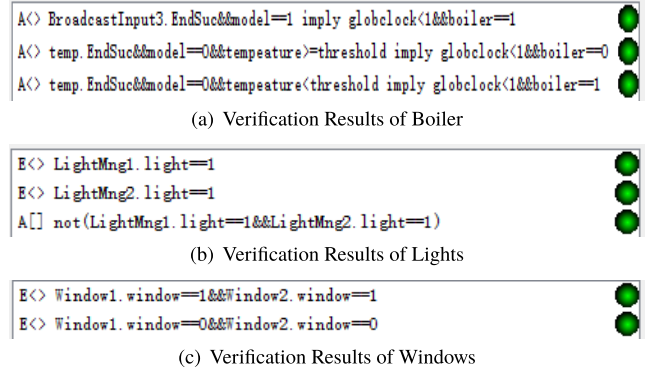


Fig. 5 Verification results

- **Property 6** is valid. It shows that two windows can be opened or closed simultaneously.

5. Conclusion and Future Work

In this paper, we enriched the CaIT calculus by introducing broadcast communications and adding explicit processes to model node mobility, the interactions between sensors (or actuators) with the environment. In order to adopt UPPAAL to explore the temporal properties of IoT networks described by the CaIT calculus, we have built a relationship between the CaIT calculus and timed automata. Then we gave a simple “smart home” to show the application of this relationship. By applying UPPAAL, we have verified six temporal properties of the “smart home”. The verification results indicate that the “smart home” can operate properly.

In the future, we will further improve our transformation performance, make it more automatic, and implement it using tools based on discrete time models. Further, we will try to apply it to more complex physical scenes and explore more interesting properties, such as the equivalence and refinement relations between IoT systems. In addition, many studies have successfully implemented the conversion between automata and fixed-point logic [33], [34], which will enable us to explore fixed-point operators more deeply. Finally, our calculus will be combined with Quality Calculus to support communications with quality predicates [35]. As a result, our calculus can capture features of IoT systems (e.g., the node mobility, and behaviours of sensors and actuators), and ensure that components work normally even when communication becomes unreliable.

References

- [1] V. Araujo, K. Mitra, S. Saguna, and C. Åhlund, “Performance evaluation of FIWARE: A cloud-based IoT platform for smart cities,” *J. Parallel Distributed Comput.*, vol. 132, pp. 250–261, Oct. 2019.
- [2] F. Alshehri and G. Muhammad, “A comprehensive survey of the Internet of Things (IoT) and AI-based smart healthcare,” *IEEE Access*, vol. 9, pp. 3660–3678, 2021.
- [3] I.U. Din, B. Ahmad, A. Almogren, H.N. Almajed, I. Mohiuddin, and J.J.P.C. Rodrigues, “Left-right-front caching strategy for vehicular networks in ICN-based Internet of Things,” *IEEE Access*, vol. 9,

- pp.595–605, 2021.
- [4] A. Krishna and G. Salaün, “Business process models for analysis of industrial IoT applications,” *IoT ’21: 11th International Conference on the Internet of Things*, pp.102–109, ACM, 2021.
 - [5] A. Krishna, M. Le Pallec, R. Mateescu, L. Noirie, and G. Salaün, “IoT Composer: Composition and deployment of IoT applications,” *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*, pp.19–22, 2019.
 - [6] R. Cleaveland, A.W. Roscoe, and S.A. Smolka, “Process algebra and model checking,” in *Handbook of Model Checking*, pp.1149–1195, Springer, 2018.
 - [7] J.A. Bergstra, A. Ponse, and S.A. Smolka, eds., *Handbook of Process Algebra*, North-Holland / Elsevier, 2001.
 - [8] A. Fehnker, R. van Glabbeek, P. Höfner, A. McIver, M. Portmann, and W.L. Tan, “Automated analysis of aadv using uppaal,” *TACAS*, pp.173–187, Springer, 2012.
 - [9] D.M. Jackson, *Logical verification of reactive software systems*, Ph.D. thesis, University of Oxford, UK, 1992.
 - [10] J. Ouaknine and J. Worrell, “Timed CSP = closed timed epsilon-automata,” *Nord. J. Comput.*, vol.10, no.2, pp.99–133, 2003.
 - [11] I. Lanese, L. Bedogni, and M.D. Felice, “Internet of things: a process calculus approach,” *Proc. 28th Annual ACM Symposium on Applied Computing*, pp.1339–1346, ACM, March 2013.
 - [12] R. Lanotte and M. Merro, “A semantic theory of the Internet of Things,” *Inf. Comput.*, vol.259, no.1, pp.72–101, April 2018.
 - [13] K.V.S. Prasad, “A calculus of broadcasting systems,” *Sci. Comput. Program.*, vol.25, no.2-3, pp.285–327, Dec. 1995.
 - [14] C. Ene and T. Muntean, “A broadcast-based calculus for communicating systems,” *Proceedings 15th International Parallel and Distributed Processing Symposium. IPDPS 2001*, pp.1516–1525, 2001.
 - [15] S. Nanz and C. Hankin, “A framework for security analysis of mobile wireless networks,” *Theor. Comput. Sci.*, vol.367, no.1-2, pp.203–227, Nov. 2006.
 - [16] I. Lanese and D. Sangiorgi, “An operational semantics for a calculus for wireless systems,” *Theor. Comput. Sci.*, vol.411, no.19, pp.1928–1948, April 2010.
 - [17] A. Singh, C.R. Ramakrishnan, and S.A. Smolka, “A process calculus for mobile ad hoc networks,” *Sci. Comput. Program.*, vol.75, no.6, pp.440–469, June 2010.
 - [18] C.A.R. Hoare, *Communicating Sequential Processes*, Prentice-Hall, 1985.
 - [19] B. Aman and G. Ciobanu, “Real-time migration properties of rTiMo verified in Uppaal,” *Software Engineering and Formal Methods - 11th International Conference, SEFM 2013*, pp.31–45, Springer, 2013.
 - [20] S. Cattani and M.Z. Kwiatkowska, “A refinement-based process algebra for timed automata,” *Formal Aspects Comput.*, vol.17, no.2, pp.138–159, Aug. 2005.
 - [21] R. Alur and D.L. Dill, “A theory of timed automata,” *Theor. Comput. Sci.*, vol.126, no.2, pp.183–235, April 1994.
 - [22] G. Behrmann, A. David, and K.G. Larsen, “A tutorial on Uppaal,” *SFM-RT 2004, Lect. Notes Comput. Sci.*, vol.3185, pp.200–236, Springer, 2004.
 - [23] J. Bengtsson and W. Yi, “Timed automata: Semantics, algorithms and tools,” *ACPN 2003, Lect. Notes Comput. Sci.*, vol.3098, pp.87–124, Springer, 2003.
 - [24] R. Milner, J. Parrow, and D. Walker, “A calculus of mobile processes, I,” *Inf. Comput.*, vol.100, no.1, pp.1–40, Sept. 1992.
 - [25] M. Abadi and A.D. Gordon, “A calculus for cryptographic protocols: The spi calculus,” *Inf. Comput.*, vol.148, no.1, pp.1–70, Jan. 1999.
 - [26] M. Abadi, B. Blanchet, and C. Fournet, “The applied pi calculus: Mobile values, new names, and secure communication,” *J. ACM*, vol.65, no.1, pp.1:1–1:41, Oct. 2018.
 - [27] J. Baeten, T. Basten, and M. Reniers, *Process Algebra: Equational Theories of Communicating Processes*, 07 2014.
 - [28] J. He, “Process simulation and refinement,” *Formal Aspects Com-*

put., vol.1, no.3, pp.229–241, March 1989.

- [29] T.A. Henzinger, X. Nicollin, J. Sifakis, and S. Yovine, “Symbolic model checking for real-time systems,” *Inf. Comput.*, vol.111, no.2, pp.193–244, June 1994.
- [30] S. Liu and N. Yoshiura, “Model checking of TTCAN protocol using UPPAAL,” *Computational Science and Its Applications - ICCSA 2018, Lect. Notes Comput. Sci.*, vol.10963, pp.550–564, Springer, July 2018.
- [31] K.G. Larsen, P. Pettersson, and W. Yi, “UPPAAL in a nutshell,” *Int. J. Softw. Tools Technol. Transf.*, vol.1, no.1-2, pp.134–152, Feb. 1997.
- [32] <https://github.com/Cnn-c/UPPAAL-and-CaIT>.
- [33] Y. Venema, “Automata and fixed point logics for coalgebras,” *CMCS 2004, Electron. Notes Theor. Comput. Sci.*, vol.106, pp.355–375, Elsevier, Dec. 2004.
- [34] Y. Venema, “Automata and fixed point logic: A coalgebraic perspective,” *Inf. Comput.*, vol.204, no.4, pp.637–678, April 2006.
- [35] H.R. Nielson, F. Nielson, and R. Vigo, “A calculus for quality,” *FACS 2012, Lect. Notes Comput. Sci.*, vol.7684, pp.188–204, Springer, 2012.

Appendix: Operational Semantics of the Original CaIT

Here, we list two transition rules of the original CaIT calculus (see [12] for complete semantics rules), which have been used in the proof of Theorem 1.

$$\begin{aligned}
 (\text{PCom}) \quad & \frac{M \xrightarrow{c!(v)@m} M', N \xrightarrow{c?(v)@n} N', \text{Dis}(n, m) \leq \text{Rng}(c)}{M \parallel N \xrightarrow{\tau} M' \parallel N'} \\
 (\text{ParN}) \quad & \frac{M \xrightarrow{v} M', v \neq \sigma}{M \parallel N \xrightarrow{v} M' \parallel N} \\
 (\text{TimeParN}) \quad & \frac{M \xrightarrow{\sigma} M', N \xrightarrow{\sigma} N', M \parallel N \xrightarrow{\sigma}}{M \parallel N \xrightarrow{\sigma} M' \parallel N'}
 \end{aligned}$$



Ningning Chen is currently a Ph.D. candidate in Shanghai Key Laboratory of Trustworthy Computing, East China Normal University, Shanghai. Her research interests contain Hoare logic, formal methods, process algebra and IoT.



Huibiao Zhu is currently a professor in East China Normal University. He earned his Ph.D. degree in formal methods from London South Bank University, London, in 2005. During these years, he has studied various semantics and their linking theories for Verilog, SystemC, web services and probability system. He was the Chinese PI of the Sino-Danish Basic Research Center IDEA4CPS.